

We can see that these mathematical criteria for judging PRNGs have some relation to each other: the lack of a long or full period can be the cause of a lack of uniform distribution. From a more practical perspective, these mathematical properties are important only because they cause our roulette app to lose money. To state it more generally, the only important test of a PRNG is whether a pattern can be detected in it.

Unfortunately, the ability to detect a pattern is hard to pin down concisely in mathematical or scientific language. So we look for long, full period and uniform distribution as markers that give us a hint about pattern detection. But of course, they're not the only clues that enable us to detect a pattern. Consider the LCG denoted by `list_random(1,1,37)`. This outputs the following list:

---

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22,
23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 0, 1]
```

---

This has a long period (37), a full period (37), and a uniform distribution (each number has likelihood  $1/37$  of being output). However, we can still detect a pattern in it (the number goes up by 1 every round until it gets to 36, and then it repeats from 0). It passes the mathematical tests we devised, but it definitely fails the roulette test.

## ***The Diehard Tests for Randomness***

There is no single silver-bullet test that indicates whether there's an exploitable pattern in a PRNG. Researchers have devised many creative tests to evaluate the extent to which a collection of random numbers is resistant to pattern detection (or in other words can pass the roulette test). One collection of such tests is called the *Diehard* tests. There are 12 Diehard tests, each of which evaluates a collection of random numbers in a different way. Collections of numbers that pass every Diehard test are deemed to have a very strong resemblance to true randomness. One of the Diehard tests, called the *overlapping sums test*, takes the entire list of random numbers and finds sums of sections of consecutive numbers from the list. The collection of all these sums should follow the mathematical pattern colloquially called a *bell curve*. We can implement a function that generates a list of overlapping sums in Python as follows: